

Python e a Matemática de Dados Financeiros: Conceitos e Aplicações

Tratando Dados Faltantes

Outubro de 2025

1 Introdução

Nas atividades práticas é comum lidar com conjuntos de dados que contém valores faltantes. Isso pode ocorrer por falhas de coleta, erros de processamento, ou até mesmo porque o dado naquele cenário está de fato inexistente. A biblioteca `pandas`, do Python, fornece ferramentas para identificar e tratar esses casos.

Antes de entrarmos no tratamento de dados faltantes, é preciso entender o que isso significa em termos computacionais. Neste momento estamos interessados em diferenciar os cenários em que temos o valor zero, o vazio, e o valor `NaN`.

O zero é um valor numérico definido e válido, indicando ausência de quantidade, e ainda que fruto de um erro é um dado legítimo. O valor vazio, como uma string vazia ou campo sem preenchimento, representa a presença de um campo sem conteúdo, muitas vezes usado para dados textuais. Já o `NaN` (Not a Number) é um valor especial usado em contextos numéricos para indicar que o dado está ausente ou inválido, como resultado de um erro de cálculo ou uma falha na coleta de dados.

Vejamos o seguinte exemplo de dataframe representando uma carteira de ativos:

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
1	PETR4	2025-09-30	38.45	100	3845.00
2	PETR4	2025-10-01	0	80	0.00
3	PETR4	2025-10-02	NaN	95	NaN
4	VALE3	2025-09-30	67.80	60	4068.00
5	VALE3	2025-10-01	68.05	50	3402.50
6	VALE3	2025-10-02	68.25	0	0.00
7	ITUB4	2025-09-30	28.90	NaN	NaN
8	ITUB4	2025-10-01	NaN	120	NaN
9	ITUB4	2025-10-02	29.00	0	0.00
10	ABEV3	2025-09-30	14.30	100	1430.00
11	ABEV3	2025-10-01	0	100	0.00
12	ABEV3	2025-10-02	NaN	NaN	NaN
13		2025-09-30	15.75	0	0.00
14		2025-10-01			
15		2025-10-02	16.00	10	160.00

É possível perceber que essa tabela contém variados tipos de dados faltantes. Vamos utilizar a partir de agora alternativas de funções do **pandas** pra corrigi-las.

2 Verificando dados faltantes

Em nosso exemplo temos uma tabela com poucos dados onde uma breve inspeção visual é suficiente para identificar quais dados estão faltantes ou incorretos. Entretanto, em situações típicas do "mundo real" as tabelas irão apresentar milhares de linhas, sendo necessário o uso das ferramentas da biblioteca para essa identificação.

2.1 Identificando Texto Vazio

```
df[df["Ativo"] == ""]
```

Aqui filtramos todos os elementos vazios da coluna *Ativo*.

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
13		2025-09-30	15.75	0	0.00
14		2025-10-01			
15		2025-10-02	16.00	10	160.00

2.2 Identificando Zeros

```
df[df["Quantidade"] == 0]
```

Podemos usar o mesmo código para filtrar os valores zero.

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
6	VALE3	2025-10-02	68.25	0	0.00
9	ITUB4	2025-10-02	29.00	0	0.00
13		2025-09-30	15.75	0	0.00

2.3 Identificando NaN

```
df[df['Valor Fechamento'].isna()]
```

No **pandas** temos a função dedicada *.isna()* para trabalhar com valores NaN que usamos em nosso código.

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
3	PETR4	2025-10-02	NaN	95	NaN
8	ITUB4	2025-10-01	NaN	120	NaN
12	ABEV3	2025-10-02	NaN	NaN	NaN

3 Removendo dados faltantes

Diante da ocorrência de um dado vazio ou incorreto, em geral duas medidas corretivas podem ser realizadas. Podemos simplesmente excluir aquela informação do dataframe que será usado para análise, ou, aplicar alguma correção sobre os dados. Veremos exemplos específicos de cada situação a seguir.

3.1 Remoção de Texto Vazio

```
df_texto_vazio_removido = df[df["Ativo"] != ""]
```

Criamos um novo dataframe que não contém os itens vazios da coluna *Ativo*.

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
1	PETR4	2025-09-30	38.45	100	3845.00
2	PETR4	2025-10-01	0	80	0.00
3	PETR4	2025-10-02	NaN	95	NaN
4	VALE3	2025-09-30	67.80	60	4068.00
5	VALE3	2025-10-01	68.05	50	3402.50
6	VALE3	2025-10-02	68.25	0	0.00
7	ITUB4	2025-09-30	28.90	NaN	NaN
8	ITUB4	2025-10-01	NaN	120	NaN
9	ITUB4	2025-10-02	29.00	0	0.00
10	ABEV3	2025-09-30	14.30	100	1430.00
11	ABEV3	2025-10-01	0	100	0.00
12	ABEV3	2025-10-02	NaN	NaN	NaN

3.2 Remoção de Zeros

Relembramos que apesar do valor zero ser um valor legítimo, uma ação fechar o dia com o valor zero é provavelmente algum erro de processamento. Se no contexto da análise for conveniente remover essas linhas podemos usar o método semelhante ao anterior:

```
df_zero_removido = df[df["Valor Fechamento"] != 0]
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
1	PETR4	2025-09-30	38.45	100	3845.00
3	PETR4	2025-10-02	NaN	95	NaN
4	VALE3	2025-09-30	67.80	60	4068.00
5	VALE3	2025-10-01	68.05	50	3402.50
6	VALE3	2025-10-02	68.25	0	0.00
7	ITUB4	2025-09-30	28.90	NaN	NaN
8	ITUB4	2025-10-01	NaN	120	NaN
9	ITUB4	2025-10-02	29.00	0	0.00
10	ABEV3	2025-09-30	14.30	100	1430.00

12	ABEV3	2025-10-02	NaN	NaN	NaN	
13		2025-09-30	15.75	0	0.00	
14		2025-10-01				
15		2025-10-02	16.00	10	160.00	

Perceba que removemos as linhas do dataframe inicial, porém também poderíamos remover do dataframe que criamos ao remover os campos de texto em branco, e dessa forma ter um dataframe mais limpo.

3.3 Remoção de valores NaN

Assim como no cenário anterior, quando estamos tratando de dados NaN a biblioteca **pandas** possui uma função específica que possibilita remover todas as linhas com NaN independente da coluna:

```
df_nan_removido = df.dropna()
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total	
----	-----	-----	-----	-----	-----	
1	PETR4	2025-09-30	38.45	100	3845.00	
2	PETR4	2025-10-01	0	80	0.00	
4	VALE3	2025-09-30	67.80	60	4068.00	
5	VALE3	2025-10-01	68.05	50	3402.50	
6	VALE3	2025-10-02	68.25	0	0.00	
9	ITUB4	2025-10-02	29.00	0	0.00	
10	ABEV3	2025-09-30	14.30	100	1430.00	
11	ABEV3	2025-10-01	0	100	0.00	
13		2025-09-30	15.75	0	0.00	
14		2025-10-01				
15		2025-10-02	16.00	10	160.00	

4 Tratando dados faltantes

Em alguns cenários não estamos interessados em simplesmente remover as linhas com dados faltantes, mas sim, queremos corrigir esses dados de alguma maneira. Podemos inserir os valores manualmente, ou utilizar alguma estratégia de preenchimento como usar a média dos valores, repetir algum valor, ou ainda, recalcular alguma coluna.

Vamos considerar para esse exemplo um cenário em que estamos trabalhando apenas com os dados de ITUB4:

```
df_itub4 = df[df["Ativo"] == "ITUB4"]
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total	
----	-----	-----	-----	-----	-----	
7	ITUB4	2025-09-30	28.90	NaN	NaN	
8	ITUB4	2025-10-01	NaN	120	NaN	
9	ITUB4	2025-10-02	29.00	0	0.00	

Temos na coluna *Data Fechamento* dados para os dias 30 e 02, mas não para o dia 01. Se sabemos que o valor do dia faltante, temos alternativas para alterar esse NaN.

4.1 Preenchimento com Forward Fill

Neste método, todos os NaN que seguirem em sequência a partir de um determinado valor são alterados para esse valor, até que a sequência encontre outro valor válido.

```
df_forward_fill = df_itub4.ffill()
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
7	ITUB4	2025-09-30	28.90	NaN	NaN
8	ITUB4	2025-10-01	28.90	120	NaN
9	ITUB4	2025-10-02	29.00	0	0.00

4.2 Preenchimento com Backward Fill

Este método é análogo ao anterior, porém ele repetirá os valores "de baixo para cima"

```
df_backward_fill = df_itub4.bfill()
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
7	ITUB4	2025-09-30	28.90	120	0.00
8	ITUB4	2025-10-01	29.90	120	0.00
9	ITUB4	2025-10-02	29.00	0	0.00

Perceba que como não especificamos qual coluna desejariamos aplicar o método, todas as colunas sofreram a alteração.

4.3 Preenchimento com Valor Estatístico

Em algumas situações pode fazer sentido que o valor que irá substituir o dado faltante possa vir de algum cálculo ou de resultado estatístico. Por didática, vamos supor que a média dos valores de fechamento substituiria satisfatoriamente o valor faltante em nossa análise:

```
df_itub4["Valor Fechamento"] = df_itub4["Valor Fechamento"].fillna(df_itub4["Valor Fechamento"].mean())
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
7	ITUB4	2025-09-30	28.90	NaN	NaN
8	ITUB4	2025-10-01	28.95	120	NaN
9	ITUB4	2025-10-02	29.00	0	0.00

4.4 Preenchimento com Valor Fixo

Também temos a opção de alterar especificamente um campo com um valor fixo, caso esse seja de nosso conhecimento e interesse. Uma opção é substituir todos os NaN de uma determinada coluna pelo valor fixo.

```
df_itub4["Valor Fechamento"] = df_itub4["Valor Fechamento"].fillna(28.80)
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
7	ITUB4	2025-09-30	28.90	NaN	NaN
8	ITUB4	2025-10-01	28.80	120	NaN
9	ITUB4	2025-10-02	29.00	0	0.00

Perceba que nossa coluna continha apenas um NaN, e que caso houvessem outros, com esse código todos seriam substituídos pelo mesmo valor fixado.

Existem alternativas para alterar de forma única uma célula específica do dataframe e apresentamos uma delas a seguir. Por comodidade vamos utilizar o dataframe já corrigido acima e corrigir agora o NaN da coluna *Quantidade*.

```
df_itub4.loc[(df_itub4['Data Fechamento'] == '2025-09-30') & (df_itub4['Quantidade'].isna()), 'Quantidade'] = 120
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
7	ITUB4	2025-09-30	28.90	120	NaN
8	ITUB4	2025-10-01	28.80	120	NaN
9	ITUB4	2025-10-02	29.00	0	0.00

Caso você esteja se perguntando sobre a *Quantidade* zero na linha do dia 02, como comentamos algumas vezes ao longo deste texto, o valor zero não é necessariamente um erro, e nesse caso pode significar que ao longo do dia todos os ativos foram vendidos.

É importante notar que alterar os dados faltantes manualmente pode ser uma tarefa inviável em dataframes com muitos dados. Em bases grandes com muitos erros é preferível buscar a solução na fonte da coleta ou processamento para evitar que se repitam, do que tratar os dados faltantes.

4.5 Calculando uma coluna

No dataframe anterior sobraram valores faltantes do tipo NaN apenas na coluna *Total*. Essa coluna é uma multiplicação entre os valores da coluna *Quantidade* com os valores da coluna *Valor Fechamento*. Como temos os valores dessas duas colunas já corrigidos, podemos simplesmente recalculá-la na coluna *Total*.

```
df_itub4['Total'] = df_itub4['Quantidade'] * df_itub4['Valor Fechamento']
```

ID	Ativo	Data Fechamento	Valor Fechamento	Quantidade	Total
7	ITUB4	2025-09-30	28.90	120	3468.0
8	ITUB4	2025-10-01	28.80	120	3456.0
9	ITUB4	2025-10-02	29.00	0	0.00

5 Conclusão

O `pandas` oferece flexibilidade para aplicar estratégias diferentes conforme a natureza dos dados e os objetivos da análise. Para outros tratamentos ou formas diferentes de usar os tratamentos aqui apresentamos, sugerimos a consulta na documentação oficial da biblioteca.